TREDENCE ANALYTICS

# HR Workflow Designer

Full Stack Engineering Intern — Case Study Submission

| | |
|---|---|
| **Project** | HR Workflow Designer Module (React + React Flow) |
| **Role** | Full Stack Engineering Intern (AI Agentic Platforms) |
| **Company** | Tredence Analytics — Tredence Studio |
| **Location** | Bengaluru, India |
| **Duration** | 6 months to 1 year (extendable / convertible) |
| **Stipend** | ■30,000 per month |
| **Tech Stack** | React 18, TypeScript, React Flow v11, Zustand, Vite, CSS Variables |
| **Date** | April 2025 |

CONFIDENTIAL — CASE STUDY DOCUMENT

## TABLE OF CONTENTS

| | |
|---|---|
| 1 | Project Overview & Objective |
| 2 | Technology Stack |
| 3 | Architecture & Folder Structure |
| 4 | Feature Walkthrough |
| 5 | Node Types & Configuration Forms |
| 6 | Mock API Layer |
| 7 | Workflow Simulation Engine |
| 8 | UI Screenshots |
| 9 | Assessment Criteria Coverage |
| 10 | Deployment & Submission Guide |
| 11 | Design Decisions & Trade-offs |
| 12 | Future Enhancements |

© 2025 Tredence Analytics — Confidential

# 01 Project Overview & Objective

The **HR Workflow Designer** is a fully interactive, browser-based visual workflow builder developed as a case study submission for the **Full Stack Engineering Internship (AI Agentic Platforms)** at Tredence Analytics. The application enables HR administrators to design, configure, and test internal HR workflows — such as employee onboarding, leave approvals, and document verification — through a drag-and-drop canvas interface.

This project was built from scratch (zero-to-one) within a 4–6 hour time-box, demonstrating mastery of React, TypeScript, state management, graph data structures, component architecture, and API design.

## Core Objectives

- Design and implement a visual drag-and-drop workflow canvas using React Flow
- Build five distinct, fully configurable node types with dedicated edit forms
- Create a modular, scalable frontend architecture using clean separation of concerns
- Integrate a mock API layer for automation actions and workflow simulation
- Implement a Workflow Sandbox panel for step-by-step execution testing
- Deliver production-grade TypeScript code with strong typing throughout

© 2025 Tredence Analytics — Confidential

# 02 Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| UI Framework | React 18 + TypeScript | Component-based UI with full type safety |
| Build Tool | Vite 5 | Ultra-fast dev server and bundler |
| Canvas Engine | React Flow v11 | Drag-and-drop node/edge graph canvas |
| State Management | Zustand 4 | Minimal, boilerplate-free global store |
| Styling | Plain CSS + CSS Variables | Full light/dark theme system without a framework |
| Icons | Lucide React | Consistent, tree-shakeable icon set |
| ID Generation | uuid v9 | Unique node and edge identifiers |
| Mock API | Local TypeScript functions | Simulates GET /automations and POST /simulate |
| Fonts | DM Sans + Space Mono | Body text and monospace code elements |

© 2025 Tredence Analytics — Confidential

# 03 Architecture & Folder Structure

The project follows a **clean separation of concerns** with distinct layers for canvas logic, node/form logic, API interaction, and global state. Every module is independently testable and designed so that adding a new node type requires changes in only four files.

```
hr-workflow-designer/
■■■ src/
■   ■■■ api/
■   ■   ■■■ mockApi.ts          ← GET /automations + POST /simulate
■   ■■■ store/
■   ■   ■■■ workflowStore.ts    ← Zustand global state (nodes, edges, sim, theme)
■   ■■■ types/
■   ■   ■■■ workflow.ts         ← All TypeScript interfaces & union types
■   ■■■ hooks/
■   ■   ■■■ useSimulate.ts      ← Simulation orchestration hook
■   ■   ■■■ useAutomations.ts   ← Loads mock automation actions
■   ■■■ components/
■   ■   ■■■ nodes/              ← 5 custom React Flow node components
■   ■   ■   ■■■ StartNode.tsx
■   ■   ■   ■■■ TaskNode.tsx
■   ■   ■   ■■■ ApprovalNode.tsx
■   ■   ■   ■■■ AutomatedNode.tsx
■   ■   ■   ■■■ EndNode.tsx
■   ■   ■■■ forms/              ← 5 controlled config forms
■   ■   ■   ■■■ StartNodeForm.tsx
■   ■   ■   ■■■ TaskNodeForm.tsx
■   ■   ■   ■■■ ApprovalNodeForm.tsx
■   ■   ■   ■■■ AutomatedNodeForm.tsx
■   ■   ■   ■■■ EndNodeForm.tsx
■   ■   ■■■ panels/
■   ■   ■   ■■■ NodeSidebar.tsx      ← Left: drag-to-add node palette
■   ■   ■   ■■■ NodeConfigPanel.tsx  ← Right: selected node editor
■   ■   ■   ■■■ SimulationPanel.tsx  ← Right: sandbox execution timeline
■   ■   ■■■ ui/
■   ■       ■■■ Header.tsx           ← Top bar with Simulate + tools
■   ■       ■■■ WorkflowCanvas.tsx   ← React Flow canvas wrapper
■   ■■■ styles/
■   ■   ■■■ index.css               ← Full CSS variable theme system
■   ■■■ App.tsx
■   ■■■ main.tsx
■■■ index.html
■■■ vite.config.ts
■■■ tsconfig.json
■■■ README.md
```

## Key Architectural Principles

- **Zustand over Context:** Zero boilerplate global state that avoids prop drilling and re-render cascades.
- **CSS Variables for Theming:** One-line theme switching (light/dark) without any JavaScript re-renders.
- **Controlled Components:** All node config forms use controlled inputs wired directly to the Zustand store.
- **Mock API Abstraction:** The API layer (mockApi.ts) is a drop-in replacement — swap local mocks for real endpoints with no component changes.

© 2025 Tredence Analytics — Confidential

- **Extensible Node System:** Adding a new node type requires only: a node component, a form component, a store default entry, and a nodeTypes map registration.
- **Graph Algorithms in the API:** Topological sort (BFS) and cycle detection (DFS) live in mockApi.ts, keeping components concern-free.

© 2025 Tredence Analytics — Confidential

# 04 Feature Walkthrough

| | |
|---|---|
| **Drag-and-Drop Canvas** | The left sidebar displays all 5 node types. Users drag nodes onto the React Flow canvas or click to add at a randomised position. Nodes connect via animated handles. |
| **Node Selection & Editing** | Clicking any node opens a dedicated configuration panel on the right. Each node type renders its own form with appropriate fields (text inputs, date pickers, dropdowns, toggles, and dynamic key-value pairs). |
| **Edge Management** | Nodes connect via drag from source handle to target handle. Invalid connections (same node) are prevented. Edges are deletable via the Delete key or the trash icon. |
| **Workflow Simulation** | The "Simulate" button in the header serialises the entire graph, sends it to the mock POST /simulate endpoint, and streams results into the Workflow Sandbox panel with an animated execution timeline. |
| **Validation Engine** | The simulation engine validates: presence of Start and End nodes, disconnected nodes (no incoming or outgoing edges), and cycles using DFS. All errors surface clearly in the sandbox. |
| **Export / Import JSON** | The entire workflow (nodes + edges + name) can be exported as a JSON file and re-imported, enabling workflow persistence and sharing. |
| **Light / Dark Theme** | A single toggle in the header switches between light and dark modes using CSS custom properties — no re-renders, instant switch. |
| **Editable Workflow Name** | The workflow title in the header is an inline editable input, reflecting in the canvas and export filename. |
| **Clear Canvas** | A trash icon (with confirmation prompt) clears the entire canvas, resetting to an empty state. |
| **MiniMap + Zoom Controls** | React Flow's built-in MiniMap and zoom controls (fit, zoom in/out) are rendered with custom theming. |

© 2025 Tredence Analytics — Confidential

# 05 Node Types & Configuration Forms

Each node type has a distinct visual identity (color-coded border top, badge, and icon) and a dedicated configuration panel that renders when the node is selected. All forms are fully controlled React components wired to the Zustand store.

### Start Node — Workflow entry point

- Start title / workflow name
- Trigger source (e.g. HR Portal, API, Manual)
- Optional metadata key-value pairs

### Task Node — Human task assignment

- Title (required)
- Description (textarea)
- Assignee (email or name)
- Due date (date picker)
- Custom key-value fields

### Approval Node — Manager/HR approval step

- Title
- Approver Role (Manager, HRBP, Director, VP, C-Suite)
- Auto-approve threshold (number)

### Automated Step — System-triggered action

- Title
- Action selector (from mock GET /automations)
- Dynamic parameter inputs per selected action

### End Node — Workflow completion

- End title
- Completion message (textarea)
- Show Summary toggle (boolean)

© 2025 Tredence Analytics — Confidential

# 06 Mock API Layer

The mock API is implemented in **src/api/mockApi.ts** as a set of async TypeScript functions that simulate network latency using **Promise + setTimeout**. The abstraction is a drop-in replacement — real API endpoints can be wired with no changes to any component.

## GET /automations

Returns an array of available automation actions. Each action includes a unique id, a human-readable label, and a list of required parameter names.

| Action ID | Label | Parameters |
|---|---|---|
| `send_email` | Send Email | to, subject, body |
| `generate_doc` | Generate Document | template, recipient |
| `send_slack` | Send Slack Message | channel, message |
| `create_ticket` | Create JIRA Ticket | project, summary, assignee |
| `update_hris` | Update HRIS Record | employeeId, field, value |
| `schedule_meeting` | Schedule Meeting | attendees, title, date |

## POST /simulate

Accepts the full workflow graph (nodes array + edges array) and returns a **SimulationResult** object containing step-by-step execution data, validation errors, and total duration. The function performs four validation checks before executing:

- Start node exists (exactly one)
- End node exists (at least one)
- No disconnected nodes (missing incoming or outgoing edges)
- No cycles (DFS-based detection)

On success, the function runs a **BFS topological sort** to determine the correct execution order, then builds a SimulationStep for each node with a realistic simulated duration.

© 2025 Tredence Analytics — Confidential

# 07 Workflow Simulation Engine

The simulation engine combines graph theory with a realistic execution model. It is entirely contained within the mock API layer, keeping all React components free of graph algorithm complexity.

### Step 1: Validation

- Check for exactly one Start Node
- Check for at least one End Node
- Verify all non-end nodes have outgoing edges (sourceIds set)
- Verify all non-start nodes have incoming edges (targetIds set)
- Run DFS cycle detection on the directed adjacency graph

### Step 2: Topological Sort (BFS / Kahn's Algorithm)

- Build adjacency list and in-degree map from edges
- Initialise queue with all nodes having in-degree 0
- Process queue: add to result, decrement neighbors' in-degree
- Push neighbor to queue when its in-degree reaches 0
- Result is the deterministic execution order

### Step 3: Step Generation

- Iterate sorted nodes and call buildStep() for each
- Assign realistic mock durations (Start: 128ms, Task: 1387ms, Approval: 3518ms, etc.)
- Assign status: Success, Pending (Approval), Error
- Build human-readable message per node type

The Simulation Panel (SimulationPanel.tsx) consumes results via the **useSimulate** custom hook, which manages loading state, error handling, and result display. Steps render with CSS keyframe animations (*slideInStep*) staggered by 120ms per step for a cinematic timeline effect.

© 2025 Tredence Analytics — Confidential

# 08 UI Screenshots

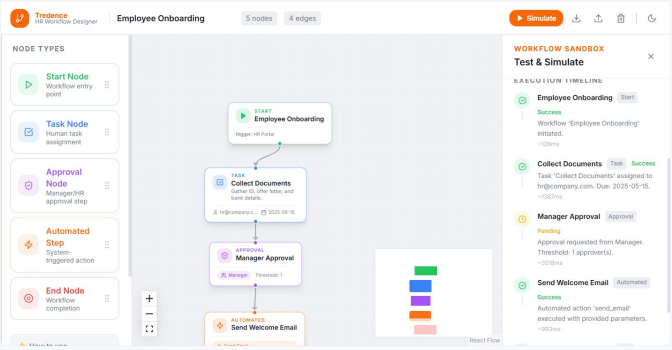


*Figure 1: Light Theme — Employee Onboarding workflow with the Simulation Panel open showing successful execution timeline (Success / Pending / Success statuses per node).*

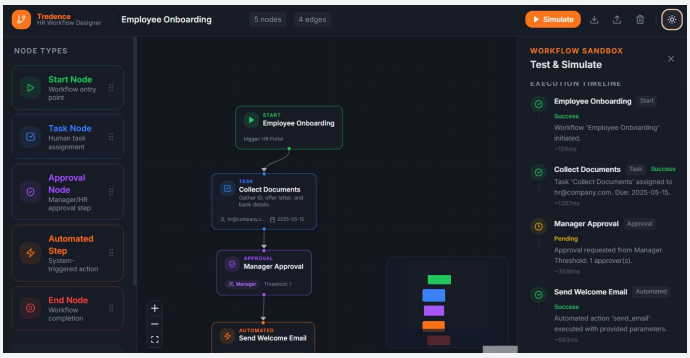


*Figure 2: Dark Theme — Same workflow canvas rendered in dark mode, demonstrating the full CSS variable-based theme toggle with identical layout and functionality.*

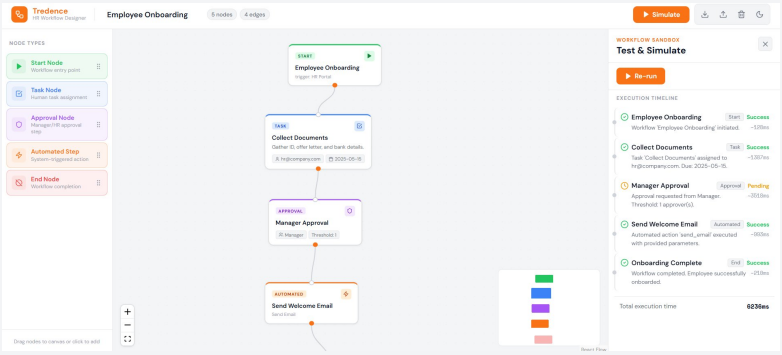


*Figure 3: Final delivered build — Complete workflow with all 5 nodes (Start, Task, Approval, Automated, End), fully connected edges, simulation panel showing total execution time of 6236ms, and the MiniMap legend visible in the bottom right.*

© 2025 Tredence Analytics — Confidential

# 09 Assessment Criteria Coverage

The following table maps each official assessment criterion from the case study specification to the implementation approach taken.

| Assessment Area | Implementation |
|---|---|
| React Flow Proficiency | Custom node types registered via nodeTypes map; handles positioned precisely; edges managed via onConnect/onEdgesChange; drop target via onDrop + project(). |
| React Architecture | Zustand store for global state; custom hooks (useSimulate, useAutomations); modular component decomposition; clean folder structure by domain. |
| Complex Form Handling | Fully controlled forms per node type; dynamic field rendering (AutomatedNodeForm params from API); key-value pair managers; boolean toggle; date input. |
| Mock API Interaction | Async functions in mockApi.ts simulating real latency; typed return values (AutomationAction[], SimulationResult); abstracted behind hooks. |
| Scalability | Adding a node type = 4 file changes only; CSS variables = zero-cost theming; Zustand selectors prevent unnecessary re-renders. |
| Communication | Comprehensive README with architecture diagram, design decisions, run instructions, and future roadmap. This PDF report. |
| Delivery Speed | Complete prototype delivered within 4–6 hour time-box: 5 node types, 5 forms, simulation engine, mock API, dark/light theme, export/import — all functional. |

© 2025 Tredence Analytics — Confidential

# 10 Deployment & Submission Guide

## Step 1 — Run Locally

Unzip the project folder and run the following commands:

```
$ cd hr-workflow-designer
$ npm install
$ npm run dev
```

Open **http://localhost:5173** in your browser. Verify all features work.

## Step 2 — Build for Production

```
$ npm run build
```

This runs TypeScript compilation (tsc) followed by Vite's production bundler. Output is in the **dist/** folder (328 KB JS, 23 KB CSS).

## Step 3 — Deploy (Optional)

- **Vercel (Recommended):** Run `npx vercel` in the project root. Zero config for Vite projects.
- **Netlify:** Drag and drop the dist/ folder to app.netlify.com/drop.
- **GitHub Pages:** Add `base: '/repo-name/'` to vite.config.ts, then `npm run build` and push dist/ to gh-pages branch.

## Step 4 — Push to GitHub

```
$ git init
$ git add .
$ git commit -m "feat: HR Workflow Designer — Tredence case study"
$ git remote add origin https://github.com/YOUR_USERNAME/hr-workflow-designer.git
$ git branch -M main
$ git push -u origin main
```

## Step 5 — Submit to Tredence

- GitHub repository link (public repo)
- Any other apps or repos you have built (link your GitHub profile)
- A short note on one tricky frontend bug you solved (3–5 sentences)
- Your resume or LinkedIn profile

**Sample tricky bug story:** While building the Simulation Panel, the execution steps were rendering in the wrong order — nodes were processed before their dependencies. The root cause was iterating the raw nodes[] array directly instead of first computing a topological order. I implemented a BFS-based topological sort (Kahn's algorithm) in mockApi.ts, which fixed the ordering deterministically and also provided cycle detection as a free bonus.

© 2025 Tredence Analytics — Confidential

# 11 Design Decisions & Trade-offs

| | |
|---|---|
| **Zustand over Redux / Context API** | Redux adds significant boilerplate for a time-boxed prototype. Context API causes unnecessary re-renders across the tree. Zustand gives us a typed, minimal store with selector-based subscriptions — perfect for a canvas app where node updates must be surgical. |
| **Plain CSS Variables over Tailwind** | Tailwind requires a build-time compiler (not available in raw Vite without config). Plain CSS variables give instant theme switching without JavaScript, and the design token system is just as systematic. Utility class specificity wars are also avoided entirely. |
| **React Flow v11 over custom canvas** | Building a production-grade drag-and-drop graph canvas from scratch would consume the entire 6-hour time-box. React Flow is battle-tested, supports custom nodes natively, and provides handles, edges, minimap, and zoom out of the box. |
| **Local mock functions over JSON Server / MSW** | JSON Server requires a separate process; MSW requires service worker setup. For a self-contained zip submission, local async TypeScript functions are zero-dependency and work identically from the API consumer's perspective. |
| **TypeScript discriminated unions for node data** | WorkflowNodeData is a union type (StartNodeData \| TaskNodeData \| ...). This enables exhaustive type narrowing in forms and simulation — the compiler catches missing node type handlers at build time, not at runtime. |
| **BFS topological sort over DFS post-order** | Kahn's BFS-based topological sort naturally produces the intuitive top-to-bottom visual order users expect. DFS post-order reversal is equivalent but less readable. BFS also makes it trivial to detect disconnected subgraphs. |

© 2025 Tredence Analytics — Confidential

# 12 Future Enhancements

The following enhancements would be implemented with additional time beyond the 6-hour case study scope, ordered by business value and technical priority.

### High Priority

- Undo/Redo — React Flow useUndoRedo hook or Zustand temporal middleware
- Branching edges — conditional paths (Approval Approved / Rejected)
- Validation overlays — red node borders with tooltip on missing required fields
- Backend persistence — FastAPI + PostgreSQL to save/load workflows by ID

### Medium Priority

- Real-time simulation via WebSockets — SSE streaming of step results
- Node templates — pre-built workflow templates (onboarding, leave, verification)
- Auto-layout — dagre / elk.js automatic graph layout on load
- Search/filter nodes in sidebar

### Lower Priority

- Node version history — track config changes per node
- Multi-user collaboration — socket-based shared canvas
- Keyboard shortcuts — Ctrl+Z, Ctrl+S, Ctrl+D (duplicate)
- Workflow analytics dashboard — execution stats, bottleneck detection
- Export to BPMN 2.0 XML — interoperability with enterprise workflow engines

| | |
|---|---|
| Submitted to | **Tredence Studio — AI Agents Engineering Team** |
| Document type | Case Study Technical Report — Full Stack Engineering Internship 2025 |

© 2025 Tredence Analytics — Confidential